ML06 Functions and checks that fail loudly

Week 2, Python you need | Session 6 of 24 | Machine Learning

Today's question. How do I turn my flower rule into a reusable tool and test it?

Objective. By the end: you can write a function that returns a result, and check it against known answers with assert.

Before this session: ML05 Lists, dictionaries and loops. If you cannot explain the last example, use its error card with the instructor first.

Words for today

Word	What it means	Think of it like this
function	A named set of instructions you can use again and again.	A recipe card you reuse.
parameter	The name inside the function for the value it receives.	The blank on a form.
argument	The actual value you give the function when you use it.	What you write in the blank.
return	Sends a result back so the program can keep it.	Handing over the finished item.
assert	Stops with an error if a statement is false.	A teacher checking that an answer is right.
check	A test with a known right answer.	Weighing a bag of sugar on a trusted scale.

Block A. A function and an assert

A function names a calculation so you can use it again. Run the cell. A passing assert prints nothing.

Predict. Predict what total(2000, 3) gives and what happens when the assert passes. Write your guess on paper first.

Your guess, written before you run it

```
def total(price, quantity):
    return price * quantity

result = total(2000, 3)
print(result)
assert total(2000, 3) == 6000
```

Run it, then compare. The cell prints 6000. def starts the function. price and quantity are parameters, and the 2000 and 3 you give it are arguments. return sends the answer back. assert stops with an error when the statement is false, and a passing assert prints nothing.

Investigate.

1. What is the difference between return and print?
2. What happens if you define total but never call it?

Modify. Add an assert that total(1000, 0) equals 0.

Block B. The flower rule as a function

Your rule from session 4 becomes a function. Three asserts test it, and a loop counts it on eight flowers.

Predict. Predict how many of the eight flowers the function names correctly. Write your guess on paper first.

Your guess, written before you run it

```
def guess_species(petal_length):
    if petal_length < 2.5:
        return "setosa"
    elif petal_length < 4.8:
        return "versicolor"
    else:
        return "virginica"

assert guess_species(1.4) == "setosa"
assert guess_species(4.7) == "versicolor"
assert guess_species(6.0) == "virginica"
flowers = [[1.4, "setosa"], [1.4, "setosa"], [4.7, "versicolor"], [4.5, "versicolor"],
    [4.9, "versicolor"], [6.0, "virginica"], [5.1, "virginica"], [5.9, "virginica"]]
right = 0
for flower in flowers:
    if guess_species(flower[0]) == flower[1]:
        right = right + 1
print(right)
```

Run it, then compare. The cell prints 7. The three asserts pass without a sound. Each flower in the list holds a petal length at position 0 and a species at position 1. The function is right for seven of eight flowers, the same count you found by hand.

Investigate.

1. What does flower[0] give?
2. Why test the function on three known flowers before counting?

Modify. Add the flower [4.9, "versicolor"] again to the list and recount.

Make

Define double(number) so it returns twice its input. Check double(4), double(0) and double(-2) with assert. Save the notebook and run all cells from the top.

Save your work as ML06. Keep the worked example and put your own attempt below it. Write two sentences: what you changed and how you checked it.

Error card

You see	It means	Fix
TypeError: total() missing 1 required positional argument	You gave the function fewer values than it has parameters.	Give one argument for each parameter.
The answer shows None	The function has no return line, so nothing came back.	Add return before the value you want back.

Exit ticket

A function packages a rule, and an assert proves the rule gives a known answer.

AI use follows AI_USE_POLICY.pdf. In weeks 1 to 6 no AI-generated solutions are used for exercises, labs or assessments. Practise for 20 to 30 minutes between classes by repeating the Make task with one value changed. If you are stuck for 30 minutes, save the error and bring it to class. A file saved on your laptop has not been submitted until you use the route in START_HERE.pdf.

After this session: foundation lab F02 on the fourth day of the week.